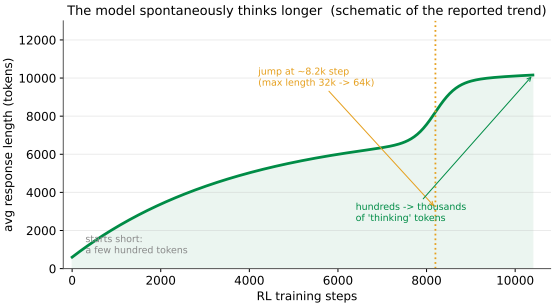
Reinforcement Learning

Part 7: reasoning models, and how they are actually trained

Part 7 of 7. RLVR, GRPO, DeepSeek-R1, and everything the field broke and fixed in 2025.

Nobody told it to think for longer



DeepSeek-R1-Zero was trained with one rule: **is the final answer correct?**

No examples of good reasoning. No reward for length, for structure, for checking your work. One number at the end of each attempt.

Over training, its answers got **longer on their own** – and inside them appeared backtracking, re-checking, and trying a second approach.

This is lecture 1's move 37, in a different domain. Nobody demonstrated the behaviour; it was **discovered**, because it was the thing that made the score go up.

Outline

Fire the annotators

For some questions, a program can grade the answer.

RLVR: the verifier replaces the human

Lecture 6 spent its whole length on a problem: nobody can write R for "a good answer", so you must learn it from human comparisons – and then guard against your policy hacking that learned model.

But for some questions, correctness is checkable

Maths	does the final answer match the reference?
Code	do the unit tests pass?
Proofs	does the formal checker accept it?
Format	is the reasoning inside the tags you asked for?

Reinforcement Learning with Verifiable Rewards (RLVR). The reward is a function you wrote, not a model you fitted.

Why this changed the field

Learned reward model

- Costs human annotation to build
- Wrong in unknown places
- **Can be hacked** – and will be
- Needs a KL leash to stay usable
- Ceiling: human judgement

Verifier

- Free, once written
- Right by construction
- **Cannot be flattered**
- Scales to millions of problems
- Ceiling: whatever is checkable

Notice what this restores: **a cheap, fast, resettable environment**. Lecture 3 said every published RL success had one, and that language models did not. A verifier *is* that environment – unlimited attempts, instant scoring, no cost per mistake.

That is why reasoning models arrived when they did.

And where it does not reach

The catch is in the name

Only *verifiable* tasks get this treatment. There is no unit test for "write a moving eulogy", "summarise this fairly", or "give good advice about my career".

So the modern stack is not RLVR *instead of* RLHF – it is both, aimed at different things:

Stage	Teaches	Signal
SFT	the format, the register	human demonstrations
Preference optimisation (DPO/RLHF)	tone, helpfulness, safety	human comparisons
RLVR	**reasoning, code, maths**	**a checker**

There is also a subtler failure: a verifier checks the **answer**, not the *reasoning*. A model can reach the right number through nonsense and be paid in full. That is what process supervision, later in this lecture, is about.

Delete the critic

PPO, minus one network, plus a very old idea.

The expensive thing in the room

Lecture 6 ended with the bill: RLHF holds **four models**, two of them being trained. The critic V_ψ is the awkward one – it is roughly policy-sized, it must be trained, and it exists for one purpose: to produce a **baseline** so the advantage has low variance.

Now recall the baseline theorem from lecture 4

You may subtract *anything that does not depend on the action* without biasing the gradient. We proved it in four lines.

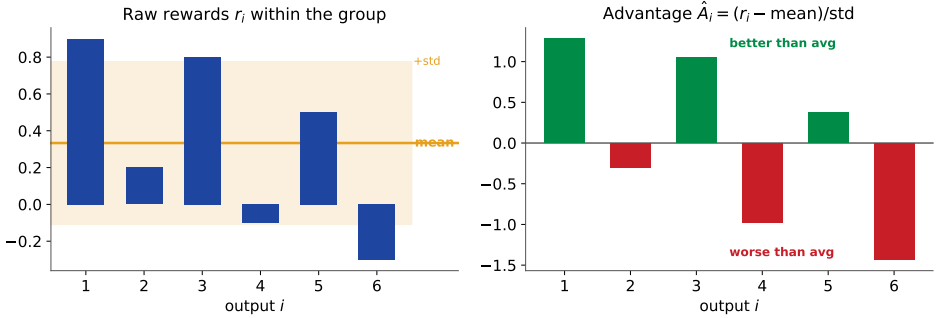
A learned $V_\psi(s)$ is one legal choice. It is not the only one.

Group Relative Policy Optimization (**GRPO**), in one sentence: for a given prompt, sample G answers, and use **the average reward of those answers** as the baseline.

It depends on the prompt, not on any individual answer – so the theorem applies, and the critic is unnecessary.

Shao et al., DeepSeekMath, 2024. Made famous by DeepSeek-R1 a year later.

Group-relative advantage



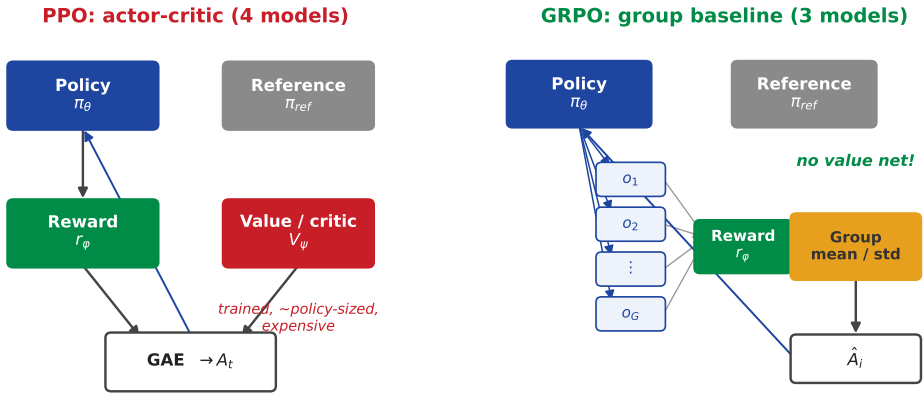
$$\hat{A}_i = \frac{R_i - \text{mean}(R_1, \ldots, R_G)}{\text{std}(R_1, \ldots, R_G)}$$

Sample G answers to the same question (typically 8–64), score them all, and **z-score within the group**. Every token of answer i gets the same $\hat{A}_i$.

"Was this answer better or worse than my other attempts at this question?" – which is the question the critic was being paid to estimate.

Four models become three

4 models → 3 models: the critic is replaced by a group-relative baseline



The trade is explicit: you stop training a value network, and you start paying for G generations per prompt instead of one. **Memory for compute** – and generation parallelises across a cluster far more happily than a second training loop does.

The GRPO objective

$$J(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(r_{i,t}(\theta)\hat{A}_i,\ \mathrm{clip}\big(r_{i,t}(\theta),1-\epsilon,1+\epsilon\big)\hat{A}_i\right) - \beta\,\mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)\right]$$

Compare it with PPO from lecture 4, term by term:

Piece	PPO	GRPO
ratio $r_{i,t}(\theta)$	$\pi_\theta/\pi_{\text{old}}$, per token	**identical**
clip	$[1-\epsilon, 1+\epsilon]$	**identical**
advantage $\hat{A}$	GAE from a learned critic	**group z-score, same for every token**
value loss	yes	**gone**
KL	often inside the reward	an explicit term in the objective

Two rows differ. That is the whole algorithm – exactly what the family tree in lecture 4 promised.

What it looked like in practice

The run that made everyone else rewrite their pipeline.

R1-Zero: no SFT at all

The experiment was deliberately extreme: take the **base** model – no instruction tuning, no demonstrations – and run GRPO against a verifier.

> Reward = **accuracy** (is the answer right?) + **format** (is the reasoning between the tags?). Nothing else.

What emerged without being asked for

- Answers grew from hundreds to thousands of tokens.
- The model began **re-checking** its own steps and **backtracking** when a line of attack failed.
- The famous "aha moment": mid-solution, it stops and reconsiders an earlier step.

Longer thinking was not the objective. It was the *strategy the optimiser found* for getting more answers right – lecture 5's test-time compute, discovered rather than designed.

And why R1-Zero was not what shipped

R1-Zero's problems

- Poor readability – the reasoning was for itself, not for you
- Language mixing mid-answer
- Strong at maths and code, unpolished everywhere else

None of these are reasoning failures. They are the predictable result of optimising a reward that never mentioned them.

SFT stages (blue) teach readable CoT; RL stages (orange) reward correctness, then broad alignment

The shipped R1 wraps the same RL in a pipeline: a small **cold-start SFT** for readability, then reasoning RL, then **rejection sampling** to build a large SFT set from the model's own best

Then everyone tried it

Four failure modes, four fixes, one year.

Failure 1: length bias

The symptom

Answers get longer and longer, and the growth is concentrated in the **wrong** answers. Models ramble, and token bills grow with no accuracy to show for it.

The cause is in the objective on the last slide. GRPO divides each answer's loss by its own length $|o_i|$. For an answer with negative advantage, dividing by a larger $|o_i|$ makes the penalty per token *smaller* – so a bad answer is punished less for being long. The optimiser notices.

Dr. GRPO (Liu et al., Sea AI Lab, 2025)

Remove the length normalisation and the standard-deviation normalisation – both are biases introduced by GRPO rather than properties of the policy gradient. Normalise by a *fixed* constant instead.

Result: much better token efficiency at the same reasoning performance, and **43.3%** on AIME 2024 with a 7B base model.

The paper's title is worth the read on its own: *Understanding R1-Zero-Like Training: A Critical*

Failures 2 and 3: the group stops teaching

Entropy collapse

The policy sharpens onto one way of answering, exploration dies, and improvement stops. Same disease as lecture 4 – and once entropy reaches zero no gradient revives it.

DAPO's fix – "Clip-Higher": decouple the clip bounds. Keep the lower bound tight, but *raise* the upper one, so low-probability tokens still have room to grow. The symmetric clip was quietly suppressing exactly the rare tokens that carry new strategies.

Groups with zero gradient

If all G answers are correct, every reward is equal, so the mean equals each reward and **every advantage is zero**. Same if all are wrong. That prompt contributes *nothing* to the update – and as the model improves, more and more prompts become all-correct.

DAPO's fix – "Dynamic Sampling": filter out prompts whose group accuracy is 0 or 1, and keep sampling until the batch is full of prompts that actually carry signal.

Failure 4, and what DAPO adds up to

Long chains dilute the signal

Averaging the loss per *sample* means a 4,000-token answer's tokens each count for a quarter as much as a 1,000-token answer's. **Token-level policy gradient loss** fixes the weighting. And answers truncated at the length limit get scored as failures they did not commit – **overlong reward shaping** stops punishing them for it.

DAPO – Decoupled Clip and Dynamic Sampling Policy Optimization (ByteDance Seed & Tsinghua, 2025)

Clip-Higher · Dynamic Sampling · Token-level loss · Overlong reward shaping

50 points on AIME 2024 with Qwen2.5-32B, beating DeepSeek-R1-Zero-Qwen-32B with **half the training steps** – and the whole system was open-sourced, which mattered as much as the score.

Note that none of the four is a new algorithm. They are four bookkeeping bugs in how the loss was averaged.

Failure 5: the ratio is at the wrong level

GRPO computes the importance ratio **per token**, but the reward it is weighting was earned by the **whole sequence**. Every token of a good answer gets the same advantage – so why correct each token's probability separately?

On **mixture-of-experts** models this becomes fatal. After an update, roughly 10% of the experts activated for a given token differ from before – so the per-token ratios swing wildly for reasons that have nothing to do with the policy being better. Training would not converge without a hack ("routing replay") to force the same experts.

Group Sequence Policy Optimization (GSPO, Zheng et al., Qwen team, 2025)

Define the importance ratio on the **sequence likelihood** (length-normalised), and clip at the sequence level. Match the unit of the ratio to the unit of the reward.

MoE training converges without routing replay, and it contributed to **Qwen3**.

A theme worth naming: **most of 2025's progress was fixing mismatches between what was rewarded and what was optimised**, not inventing new objectives.

Grading the working, not just the answer

And letting the model generate its own problems.

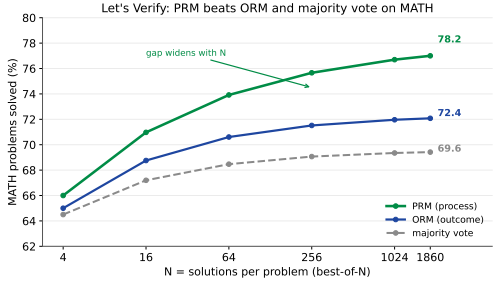
Outcome or process?

Outcome reward model (ORM)

One score for the final answer. Cheap and objective – but it pays in full for a right answer reached by luck, and gives no hint about *where* a wrong answer went wrong.

Process reward model (PRM)

A score for **every step**. Dense signal, and credit assignment becomes tractable – but it needs step-level labels, which is expensive human work.



Let's Verify Step by Step (Lightman et al., 2023) – and the gap **widens** with more candidates.

Best-of-N is search, wearing a hat

Give the model a verifier or a reward model at *inference* time: sample N answers, score them, return the best.

Look at what that is. A **policy** proposes candidates, a **value estimate** ranks them, and you spend more compute to play better.

That is AlphaGo's architecture from lecture 5, flattened to a tree of depth one.

Which suggests the obvious extension, and it is an active area: keep the tree. Score partial reasoning with a PRM, expand the promising branches, prune the rest – MCTS over chains of thought.

Self-consistency is the cheap version everyone uses: sample N chains, take the majority answer. No verifier needed, and it works because wrong reasoning disagrees with itself while correct reasoning converges.

Where the next problems come from

RLVR needs problems with checkable answers. Human-written ones run out. So the model makes its own – lecture 5's self-play, applied to questions instead of games:

- **SPIN.** The model learns to tell its own outputs from human-written ones, and improves by trying to become indistinguishable. No new annotation.
- **SPICE.** A *Challenger* mines documents to invent problems; a *Reasoner* solves them. Grounding the questions in real documents is what stops the pair drifting into hallucinated nonsense – reported $+8.9\%$ on maths and $+9.8\%$ on general reasoning.
- **Automatic curricula.** Generate problems calibrated to what the model currently gets wrong, verify with RLVR, repeat. A closed loop with no human in it.

The obvious worry is the one from lecture 5: ungrounded self-play amplifies the model's own errors. SPICE's answer – anchor to external documents – is the same instinct as offline RL's "stay where the data is".

More than one turn

Where this is all going, and where chapter 18 picks up.

Agentic RL

Everything so far treated one response as one episode. An agent that searches the web, runs code, reads the output and tries again is a **sequence of decisions with real consequences** – which is what this whole chapter was actually about.

	Single response	**Agent**
Episode	one answer	many turns of act, observe, act
State	prompt + tokens so far	+ everything the tools returned
Formally	an MDP	a **partially observable MDP** (**POMDP**)
Credit	one reward over one answer	one reward over dozens of tool calls

Credit assignment comes back with full force. The task failed – was it the third search query, the misread result, or the final summary? This is lecture 1's opening problem, and it is unsolved at this scale.

The other hard part is unglamorous: rollouts now involve real network calls, so the bottleneck is infrastructure. **verl** (ByteDance) and **NeMo Gym** (NVIDIA) exist because of this.

Chapter 18 builds the agent. This lecture explains how one gets trained.

Reward hacking, LLM edition

Lecture 5 promised this would get worse when the reward is learned. It did:

- **Sycophancy.** Agreeing with the user scores well with human raters. The model learns to agree.
- **Length and formatting.** Longer, bulleted, hedged answers won comparisons, so models produce them regardless of whether they help.
- **Verifier gaming.** With a unit-test reward, code that special-cases the tests. With an answer-matching reward, guessing a plausible number and asserting it.
- **Unfaithful reasoning.** The chain of thought is optimised for reaching a rewarded answer, not for describing what the model actually did – so it is not automatically an explanation.

The defences are the ones this chapter has been building: a **KL leash** to the reference policy, **verifiable** rewards where possible, **process** supervision where affordable, and held-out evaluation that the optimiser has never seen.

The open question

Does RL add ability, or reveal it?

Two readings of the same results, both held by serious people:

Sharpening. RL reweights behaviours the base model could already produce. Evidence: with enough samples, base models often solve problems their RL-tuned versions solve – pass@k at large k narrows the gap sharply. On this reading, RL buys reliability, not capability.

Discovery. R1-Zero developed backtracking and self-verification that were never demonstrated, and pure-RL runs beat distillation on some tasks. On this reading, RL finds genuinely new strategies.

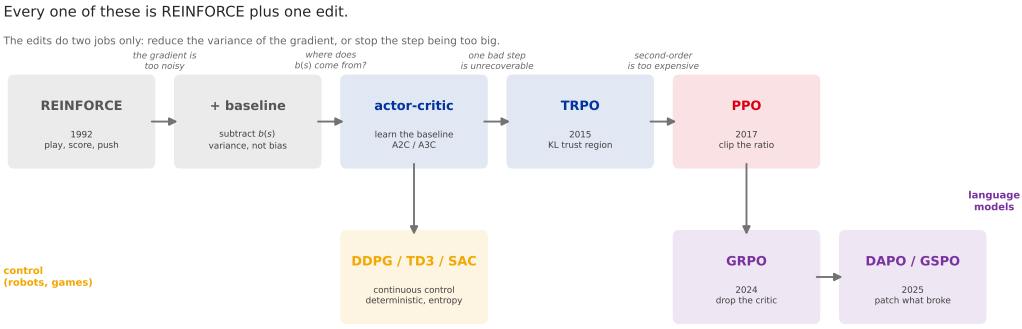
It is probably both, in different proportions per task. **The honest position is that this is unresolved** – and it is the most important open question in the area, because it decides whether scaling RL keeps paying.

Recap – the chapter, not the lecture

- **RLVR** replaces the human rater with a checker. It restores the cheap, fast, resettable environment that lecture 3 said language models did not have.
- **GRPO is PPO with the critic deleted**, its baseline replaced by the mean reward of G sampled answers – legal because of the baseline theorem you proved in lecture 4.
- **R1-Zero** shows reasoning behaviour emerging from a correctness reward alone; shipped **R1** adds human scaffolding back at the edges, exactly as AlphaGo Zero did.
- 2025 was spent fixing mismatches, not inventing objectives: **Dr. GRPO** (length bias), **DAPO** (entropy collapse, dead groups, long-chain weighting), **GSPO** (sequence-level ratios, MoE).
- **Process supervision** beats outcome supervision, and best-of-N is AlphaGo's architecture at depth one.
- **Agentic RL** is the chapter's opening problem – delayed reward, credit assignment – returning at full scale.

Where to go next: `ml/llm_training/` reads the primary sources – GRPO (DeepSeekMath), DPO, InstructGPT, DeepSeek-R1, chain-of-thought. You now have the foundations those papers assume.

Seven lectures in one picture



Bandits gave you exploration. MDPs gave you consequences. Bellman gave you the recursion.
Sampling gave you Q-learning. Networks gave you DQN. The log-derivative trick gave you policy gradients.
Everything after that is variance reduction and step-size control.